

SQL NOTES & SOLUTIONS

Authored by: Arpit Raj, LNMIIT Jaipur

1. Game Play Analysis

Table: Activity

COLUMN NAME	TYPE
player_id	int
device_id	int
event_date	date
games_played	int

(player_id, event_date) is the primary key (combination of columns with unique values) of this table.

This table shows the activity of players of some games.

Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out on someday using some device.

Write a solution to report the fraction of players that logged in again on the day after the day they first logged in, rounded to 2 decimal places. In other words, you need to determine the number of players who logged in on the day immediately following their initial login, and divide it by the number of total players.

Input: Activity table

PLAYER_ID	DEVICE_ID	EVENT_DATE	GAMES_PLAYED
1	2	2016-03-01	5
1	2	2016-03-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

Output:

FRACTION
0.33

Solution

```
SELECT ROUND(COUNT(a.player_id)/(SELECT COUNT(DISTINCT player_id) FROM
Activity),2) AS fraction
FROM Activity a

JOIN(
    SELECT player_id,MIN(event_date) AS firstlogin
    FROM Activity
    GROUP BY player_id
) b
ON a.player_id=b.player_id
WHERE a.event_date=DATE_ADD(b.firstlogin, INTERVAL 1 DAY);
```

Step 1: Find each player's first login

```
SELECT
    player_id,
    MIN(event_date)
FROM Activity
GROUP BY player_id;
```

and make a table out of it , this will be used to join main table with this table and shortlist only valid players and in step 2 just find what you need

2. Managers with at Least Five Direct Reports

Write a solution to find managers with at least five direct reports.

Return the result table in any order.

The result format is in the following example.

Example 1 Input: Employee table

ID	NAME	DEPARTMENT	MANAGERID
101	John	A	null
102	Dan	A	101
103	James	A	101
104	Amy	A	101
105	Anne	A	101
106	Ron	B	101

Output:

NAME
John

Solution 1

```
SELECT e.name
FROM Employee e

JOIN(
    SELECT managerId
    FROM Employee
    GROUP BY managerId
    HAVING COUNT(*)>=5
) m

ON e.id=m.managerId;
```

Solution 2

```
SELECT name
FROM Employee e1

WHERE(
    SELECT COUNT(*)
    FROM Employee e2
    WHERE e1.id=e2.managerId
) >=5;
```

subquery method is inefficient but works

3. Customers Who Bought All Products

Table: Customer

COLUMN NAME	TYPE
customer_id	int
product_key	int

This table may contain duplicates rows. customer_id is not NULL.
product_key is a foreign key (reference column) to Product table.

Table: Product

COLUMN NAME	TYPE
product_key	int

product_key is the primary key (column with unique values) for this table.

Write a solution to report the customer ids from the Customer table that bought all the products in the Product table.

Return the result table in any order.

The result format is in the following example.

Example 1 Input:

Customer table:

CUSTOMER_ID	PRODUCT_KEY
1	5
2	6
3	5
3	6
1	6

Product table:

PRODUCT_KEY
5
6

Output:

CUSTOMER_ID
1
3

Explanation: The customers who bought all the products (5 and 6) are customers with IDs 1 and 3.

Solution

```
SELECT customer_id
FROM Customer

GROUP BY customer_id
HAVING COUNT(DISTINCT product_key)=(
    SELECT COUNT(*)
    FROM Product
);
```

4. Market Analysis I

Table: Users

COLUMN NAME	TYPE
user_id	int
join_date	date
favorite_brand	varchar

user_id is the primary key (column with unique values) of this table.

This table has the info of the users of an online shopping website where users can sell and buy items.

Table: Orders

COLUMN NAME	TYPE
order_id	int
order_date	date
item_id	int
buyer_id	int
seller_id	int

order_id is the primary key (column with unique values) of this table.

item_id is a foreign key (reference column) to the Items table.

buyer_id and seller_id are foreign keys to the Users table.

Table: Items

COLUMN NAME	TYPE
item_id	int
item_brand	varchar

item_id is the primary key (column with unique values) of this table.

Write a solution to find for each user, the join date and the number of orders they made as a buyer in 2019.

Return the result table in any order.

Solution

```
# Write your MySQL query statement below
SELECT u.user_id AS buyer_id,u.join_date,
(
    SELECT COUNT(*)
    FROM Orders o
    WHERE u.user_id=o.buyer_id
    AND
    YEAR(o.order_date)=2019
) AS orders_in_2019

FROM Users u;
```

5. Monthly Transactions I

Table: Transactions

COLUMN NAME	TYPE
id	int
country	varchar
state	enum
amount	int
trans_date	date

id is the primary key of this table.

The table has information about incoming transactions.

The state column is an enum of type ["approved", "declined"].

Write an SQL query to find for each month and country, the number of transactions and their total amount, the number of approved transactions and their total amount.

Return the result table in any order.

The query result format is in the following example.

Example 1 Input: Transactions table

ID	COUNTRY	STATE	AMOUNT	TRANS_DATE
121	US	approved	1000	2018-12-18
122	US	declined	2000	2018-12-19
123	US	approved	2000	2019-01-01
124	DE	approved	2000	2019-01-07

Output:

MONTH	COUNTRY	TRANS_COUNT	APPROVED_COUNT	TRANS_TOTAL_AMOUNT	APPROVED_TOTAL_AMOUNT
2018-12	US	2	1	3000	1000
2019-01	US	1	1	2000	2000
2019-01	DE	1	1	2000	2000

Solution

```
# Write your MySQL query statement below
SELECT DATE_FORMAT(trans_date, '%Y-%m') AS month,
       country,
       COUNT(*) AS trans_count,
       SUM(CASE
            WHEN state='approved' THEN 1 ELSE 0
            END
       ) AS approved_count,
       SUM(amount) AS trans_total_amount,
       SUM(
            CASE
            WHEN state='approved' THEN amount ELSE 0
            END
       ) AS approved_total_amount

FROM Transactions
GROUP BY month, country;
```

6. Last Person to Fit in the Bus

Table: Queue

COLUMN NAME	TYPE
person_id	int
person_name	varchar
weight	int
turn	int

person_id column contains unique values.

This table has the information about all people waiting for a bus.

The person_id and turn columns will contain all numbers from 1 to n, where n is the number of rows in the table.

turn determines the order of which the people will board the bus, where turn=1 denotes the first person to board and turn=n denotes the last person to board.

weight is the weight of the person in kilograms.

There is a queue of people waiting to board a bus. However, the bus has a weight limit of 1000 kilograms, so there may be some people who cannot board.

Write a solution to find the person_name of the last person that can fit on the bus without exceeding the weight limit. The test cases are generated such that the first person does not exceed the weight limit.

Example 1 Input: Queue table

PERSON_ID	PERSON_NAME	WEIGHT	TURN
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2
6	John Cena	400	3
1	Winston	500	6
2	Marie	200	4

Output:

PERSON_NAME
John Cena

Solution 1

```
SELECT person_name
FROM Queue q1
WHERE(
    SELECT SUM(weight)
    FROM Queue q2
    WHERE q2.turn<=q1.turn
) <=1000

ORDER BY turn DESC
LIMIT 1;
```

Solution 2

```
# Write your MySQL query statement below
SELECT q1.person_name
FROM Queue q1
JOIN Queue q2
ON q2.turn<=q1.turn

GROUP BY q1.person_id,q1.person_name,q1.turn
HAVING SUM(q2.weight)<=1000

ORDER BY q1.turn DESC
LIMIT 1;
```